

Complete Project Development Prompt

Project Title

AI-Powered Jury Panel Mapping System

Project Objective

Develop a full-stack web application that automates the assignment of competition entries to jury panels while satisfying multiple business constraints such as jury availability, operating unit restrictions, geography, panel capacity, and scheduling.

The application must replace the existing Streamlit interface with a professional React dashboard while preserving the existing Python mapping algorithm.

The architecture should be scalable, modular, maintainable, and production-ready.

Technology Stack

Frontend

- React.js (Vite)
 - React Router
 - Axios
 - Material UI (MUI) or Ant Design
 - React Hook Form
 - React Dropzone
 - React Table (TanStack Table)
 - Recharts or Chart.js
 - React Toastify
-

Backend

- FastAPI
- Python 3.11+
- Pandas
- Openpyxl
- Uvicorn
- Pydantic

- Python Logging

Storage

Excel files (.xlsx)

No database for now.

Project Architecture

React Frontend

- |
- ├── Dashboard
- ├── Upload Module
- ├── Mapping Module
- ├── Reports
- ├── Download Output

|



FastAPI Backend

- ├── API Routes
- ├── Services
- ├── Mapping Algorithm
- ├── Validation
- ├── Excel Processing
- ├── Output Generator

|



Excel Files

Entries.xlsx

Juries.xlsx

Categories.xlsx

Time Slots.xlsx

Output.xlsx

Business Workflow

Step 1

User opens the application.

Step 2

Dashboard loads.

Display

- Total Entries
- Total Jury Members
- Available Sessions
- Previous Mapping %
- Upload Status

Step 3

User uploads required Excel files.

Required files

- Entries File
- Jury Availability File

Optional

- Category File

Validate

- File extension

- Duplicate uploads
 - Empty files
 - Invalid format
-

Step 4

Backend reads Excel files using Pandas.

Perform preprocessing.

Examples

Remove extra spaces

Normalize text

Normalize country names

Standardize date format

Standardize time slots

Handle missing values

Step 5

Generate available sessions.

Find all

Date

Time

Available juries

Step 6

For every session

Find

Available jury members.

Step 7

Create jury panels.

Rules

Minimum

2 juries

Maximum

Depends on capacity.

Step 8

Process every entry.

Check

Operating Unit constraint

Entry OU $\neq$ Jury OU

Geography compatibility

Category compatibility

Availability

No duplicate assignment

Panel capacity

Step 9

Assign entry.

Otherwise

Keep in

Unassigned list.

Step 10

Generate output.

Output includes

Assigned entries

Panel information

Jury details

Unassigned entries

Statistics

Step 11

Return

Excel file

JSON statistics

Processing logs

React Pages

1 Dashboard

Cards

Total Entries

Mapped Entries

Unmapped Entries

Total Jury Members

Mapping %

Recent Execution

Buttons

Upload Files

Run Mapping

Download Output

2 Upload Page

Drag and Drop upload.

Show

Filename

Size

Status

Progress

Remove button

3 Mapping Page

Progress Bar

Live Status

Current Entry
Current Session
Current Panel
Estimated Time
Console Logs

4 Results Page

Statistics
Pie Chart
Bar Chart
Table
Mapped
Unmapped
Search
Filter
Pagination

5 Download Page

Download
Output Excel
Logs
Reports

Sidebar

Dashboard
Upload Files
Run Mapping
Results
Reports
Settings
About

Backend Folder Structure

backend/

app/

main.py

routers/

mapping.py

upload.py

download.py

services/

excel_service.py

mapping_service.py

validation_service.py

panel_service.py

utils/

constants.py

helper.py

logger.py

models/

schemas.py

uploads/

outputs/

React Folder Structure

frontend/

src/

components/

pages/

Dashboard.jsx

Upload.jsx

Mapping.jsx

Results.jsx

Settings.jsx

services/

api.js

hooks/

context/

assets/

styles/

App.jsx

REST APIs

Upload Files

POST

/api/upload

Multipart Form Data

Response

```
{  
  "success":true  
}
```

Run Mapping

POST

/api/run-mapping

Response

```
{  
  mapping_percentage:90,  
  
  assigned:520,
```

unassigned:48,

processing_time:"18 sec"

}

Get Statistics

GET

/api/statistics

Download Output

GET

/api/download

Returns

Excel file

UI Theme

Professional Corporate Dashboard

Color Palette

Primary

Blue

Secondary

White

Accent

Green

Background

Light Gray

Cards

Rounded

Shadow

Minimal

Dashboard Widgets

Mapping %

Progress Circle

Today's Execution

Recent Uploads

Recent Outputs

Activity Timeline

Statistics Cards

Charts

Validation Rules

Reject invalid Excel

Reject empty sheets

Reject missing columns

Reject duplicate entries

Reject invalid dates

Reject duplicate jury members

Logging

Every operation should be logged.

Examples

Upload completed

Processing started

Panel created

Assignment completed

Output generated

Errors

Warnings

Error Handling

Invalid Excel

Missing Sheet

Duplicate Entries

Invalid Time Slot

No Jury Available

Panel Capacity Reached

Server Error

Performance

Support

10,000+ entries

500+ jury members

Multiple sessions

Memory optimized using Pandas

Future Scope

Role-Based Authentication

Admin Dashboard

Audit Logs

Database Support

Cloud Storage

Email Notifications

Live Monitoring

AI-based Jury Recommendation

Analytics Dashboard

Final Recommendation

If you're rebuilding the project, use:

- **Frontend:** React + Vite + Material UI
- **Backend:** FastAPI

- **Algorithm:** Reuse the existing Python mapping logic by moving it into a `mapping_service.py` module without rewriting its core logic.

This architecture is cleaner, easier to maintain, and much more suitable for future enhancements than the original Streamlit-based implementation.

1. Input files (most important)

Your prompt should explicitly state **all input files** and distinguish between user-uploaded files and internal reference files.

User-uploaded inputs:

- Entries Excel (Upload_Innovista_Entries.xlsx / TCS_Finals_Entries.xlsx)
- Jury availability Excel (multiple "Upload Update Jury_*.xlsx" files or combined jury input)

Reference/internal files (already stored on the server):

- Sub Categories
- IE.xlsx
- Category mapping
- Any lookup/reference Excel files used by the algorithm

The backend should not require the user to upload files that are static reference data.

2. Output files

Instead of simply saying "generate output", specify:

- Final mapped Excel
 - Unassigned entries sheet
 - Summary sheet
 - Statistics JSON for the dashboard
 - Processing log
-

3. Exact business constraints

Your current prompt mentions them briefly, but they should be listed explicitly:

- Operating Unit (OU) conflict check
- Jury availability
- Date availability
- Time-slot availability
- Geography compatibility

- Category compatibility (optional/future)
- Jury reuse prevention
- Duplicate entry prevention
- Maximum panel size
- Minimum panel size
- Session duration capacity
- Entry assigned only once

These come directly from the algorithm documentation.

4. Backend modules

Instead of a generic `mapping_service.py`, break the responsibilities down:

- File Upload Service
- Excel Reader
- Data Cleaning
- Data Normalization
- Constraint Engine
- Panel Generator
- Assignment Engine
- Output Generator
- Statistics Generator
- Download Service

This separation will make the project much easier to maintain.

5. API endpoints

Add APIs beyond just upload/run/download:

- POST /upload
- POST /validate-files
- POST /run-mapping
- GET /statistics
- GET /panels
- GET /unassigned

- GET /download
 - GET /logs
 - GET /health
-

6. Dashboard metrics

Include all metrics the frontend should display:

- Total Entries
 - Total Jury Members
 - Total Sessions
 - Available Sessions
 - Assigned Entries
 - Unassigned Entries
 - Mapping Percentage
 - Total Panels
 - Processing Time
 - Success/Failure Status
-

7. Data preview

Before running the algorithm, the user should be able to preview uploaded data:

- Entries table
 - Jury table
 - Number of rows
 - Missing values
 - Duplicate count
 - File validation status
-

8. Progress reporting

Instead of a static loading spinner, have the backend send progress updates such as:

- Upload completed
- Reading Excel
- Cleaning data

- Creating sessions
- Creating jury panels
- Assigning entries
- Generating output
- Completed

This greatly improves user experience.

9. Project folder structure

Add folders for:

- uploads/
 - outputs/
 - logs/
 - temp/
 - static/
 - config/
-

10. Error handling

Define expected errors clearly:

- Invalid Excel format
 - Missing required columns
 - Empty file
 - Corrupted workbook
 - No jury available
 - Panel capacity exceeded
 - Duplicate entries
 - Invalid date/time
 - Internal processing error
-

11. Configuration

Specify that configurable values should be stored in a config file or environment variables, for example:

- Maximum panel size
 - Minimum panel size
 - Allowed Excel formats
 - Upload size limit
 - Output directory
 - Logging level
-

12. Future enhancements

Your prompt can also mention:

- Authentication (Admin/User)
 - Database migration (PostgreSQL/MySQL)
 - Email notifications
 - Audit logs
 - Role-based access control
 - Analytics dashboard
 - Cloud deployment
-

One more suggestion

Instead of writing a generic software prompt, write it as a **Software Requirements Specification (SRS)**. Include:

1. Functional requirements
2. Non-functional requirements
3. User roles
4. UI wireframes
5. API contracts
6. Folder structure
7. Business rules
8. Algorithm workflow
9. Validation rules
10. Error handling
11. Deployment architecture

An SRS-style prompt is much more likely to produce a complete, production-quality React + FastAPI application with minimal back-and-forth. Based on your ZIP and the provided documentation, that would be the most complete specification for recreating this project.